INFORMATION SYSTEMS SECURITY · CHAPTER 1 · PART 2

Authentication and Access Control Techniques

How a system verifies who you are, and decides what you may do

TODAY'S LECTURE

Lecture outline

Part A

Authentication techniques

Passwords
Biometrics
Tokens
Tickets (Kerberos)
Single sign-on

Part B

Access control techniques

Discretionary (DAC)
Mandatory (MAC)
Lattice-based
Role-based (RBAC)
Rule-based and ABAC

Part C

Methodologies and administration

Centralized vs decentralized
Account administration
Least privilege
Separation of duties

RECAP FROM PART 1

What we need from Part 1

Identification → **Authentication** → **Authorization** → **Accountability**

Factor	Meaning	Examples
Type 1	Something you know	Password, PIN, passphrase
Type 2	Something you have	Smart card, token device, phone app
Type 3	Something you are	Fingerprint, iris, face, voice
Type 4	Something You Do & Somewhere You Are	Signature dynamics, Campus network, Device / terminal

Multi-factor authentication: two or more factors of **different types**. Password + PIN is still one factor type.

PART A

Identification and authentication techniques

Passwords · biometrics · tokens · tickets · single sign-on

DEFINITION

Identification and authentication

Authentication verifies the identity of a subject by comparing one or more factors against the database of valid identities (user accounts).

STEP 1

Identification

The subject **claims** an identity, e.g. a username. The identity is usually **public**.

STEP 2

Authentication

The subject **proves** it with a factor, e.g. a password. The factor must stay **private**.

The two always work together as one two-step process: neither alone gives access. A system is only as secure as the secrecy of its authentication factors.

PASSWORDS

Passwords: widely used but vulnerable

Easy to guess

Users choose passwords they can remember.

Written down

Random passwords are hard to remember, so users write them down.

Shared and reused

Given to others, or used on many sites.

Stolen

By observation, recording and playback, or database theft.

Sent in clear text

Or with weak, easily broken encryption.

Too short

Short passwords fall quickly to brute-force attacks.

PASSWORD SELECTION

Types of passwords

Type	Definition	Key point
Static	Always remains the same	Risk grows the longer it is used
Dynamic	Changes after a set interval of time or number of uses	Limits how long a stolen password works
One-time (OTP)	A dynamic password that changes every time it is used	Strongest in concept; usually generated by a token
Passphrase	A long string of characters, often a modified sentence; treated as a long password. Ex(She $ells C shells) or (BlueRiverCoffeeMorning)	Hard to brute-force, easy to remember
Composition	Two or more unrelated words joined by a number or symbol; often system-generated Ex (Book#River9)	Weak if the generating pattern is discovered
Cognitive	Answers to questions only the subject should know (e.g. your favorite color)	Weak: answers are often public; better do not use

PASSWORD POLICIES

Password policy: traditional vs current

Password policy:

system rules that restrict or dictate password characteristics, such as minimum length, minimum and maximum age, required character types, and preventing password reuse.

Traditional policy (older)	Current guidance, NIST SP 800-63B-4 (2025)
Change passwords every 30–90 days	No forced periodic changes; change only after compromise
Require upper, lower, digits and symbols	No composition rules; length matters more
Minimum of 8 characters	At least 15 if the password is the only factor; 8 with MFA
Security questions and hints	Do not use security questions or password hints
Short maximum length; paste blocked	Allow 64+ characters, paste and password managers
Not required	Reject common, expected and breached passwords

PASSWORD SECURITY

Password attacks

Attack	Definition	Main defense
Sniffing	Network traffic analysis: capturing a password to replay it	Encrypted Connection; OTPs
Password file access	Stealing the password database to crack it offline	Salted, slow hashing
Dictionary	Trying common passwords and dictionary words	Block common passwords; rate limiting; MFA
Brute force	Trying every possible character combination	Long passwords; rate limiting (online); slow hashing (offline)
Hybrid	Dictionary attack, then brute-force small changes	Long, uncommon passwords; block common passwords
Credential stuffing	Using username/password pairs leaked from one site to try logging in to other sites.	Unique passwords; MFA; breached-password checks
Password spraying	Trying one common password against many different accounts.	MFA; monitoring; rate limiting
Social engineering	Manipulating users into revealing credentials or approving unauthorized access	Training; strong MFA

PASSWORD STORAGE

Store salted hashes, never passwords



Hash:

a one-way function. The system hashes what you type and compares; it never decrypts.

The system hashes the typed pw again, then compare (New hash = Stored hash)

Salt:

a random value added to each password, so identical passwords get different hashes.x

Slow hash:

makes each guess costly, even with a stolen database.

PASSWORD SECURITY

Improving password security

Account lockout /rate limiting : disables a user account after a specified number of failed logons. It stops **online** brute-force and dictionary attacks at the logon prompt, but does not protect stolen password hashes.

- Show the time and place of the last logon, so users can report logins they do not recognize.
- Audit logon success and failure; an intrusion detection system (IDS) can alert administrators.
- Store passwords as salted hashes; never send them in clear text or by email, or show them on screen.
- Disable inactive accounts and delete accounts that are no longer used.
- If the administrator password is compromised, change every password on the system.

BIOMETRICS

Biometrics: definition and process

Biometric factor: a physiological or behavioral characteristic that is unique to a subject. Type 3: something you are.

1. **Capture** a sample
2. **Convert** to binary data
3. **Store** as a reference template
4. **Compare** a live sample with it
5. **Accept** if the match passes the threshold

Physiological (body): fingerprint, face, iris, retina, palm, hand geometry

Behavioral (actions): voice, signature dynamics, keystroke dynamics

BIOMETRICS

Identification (1:N) vs authentication (1:1)

IDENTIFICATION · ONE-TO-MANY

Biometric instead of a username

The sample is searched against **all** enrolled templates to find who the person is.

Typical use: **physical** access control, e.g. a door that opens by fingerprint alone.

AUTHENTICATION · ONE-TO-ONE

Username, then biometric

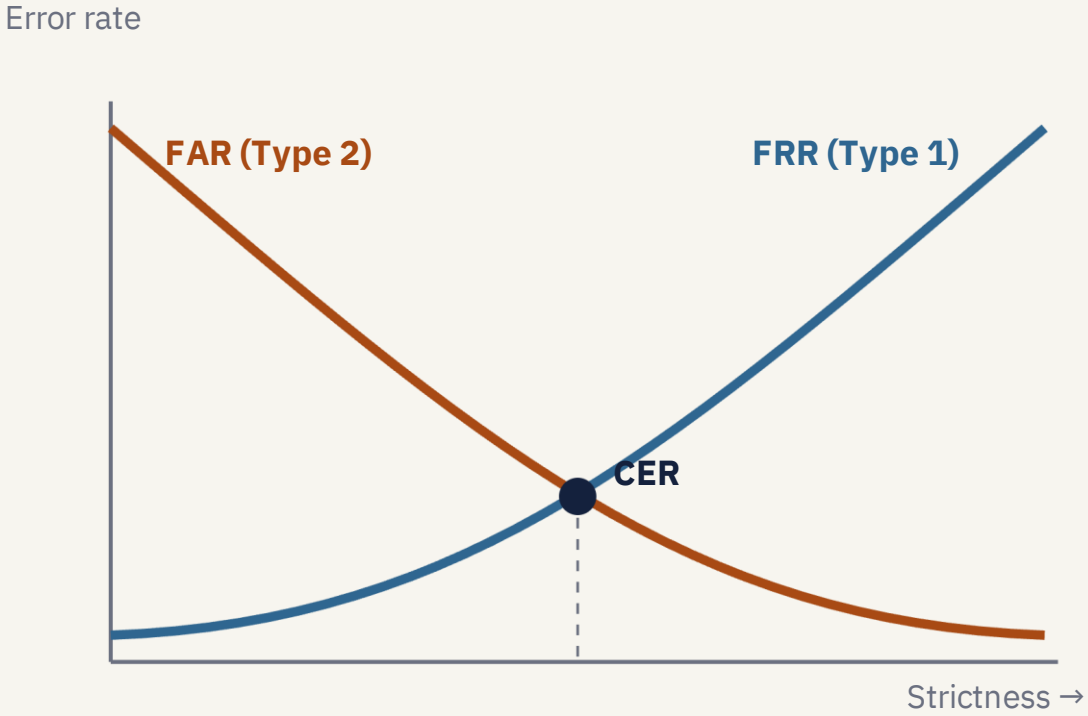
The sample is compared with the **one** stored template of the claimed identity.

Typical use: **logical** access control, e.g. logging in to a computer.

Two limits to remember: biometrics can be spoofed (fake fingers, photos), so readers machine need liveness detection; and a stolen biometric cannot be changed like a password.

BIOMETRIC FACTOR RATINGS

FRR, FAR and the crossover error rate



False rejection

A valid subject is rejected (threshold too strict ex(95%)).

FRR = false rejections ÷ valid attempts

False acceptance

An unauthorized is accepted (threshold too lenient (less strict) ex(60%)).

FAR = false acceptances ÷ unauthorized attempts

CER, also called EER (Equal Error Rate)

The point where FRR = FAR.

When devices are tested under the same conditions, the **lowest CER** is the most accurate.

BIOMETRIC REGISTRATION AND USAGE

Enrollment, throughput and acceptance

Enrollment (registration)

Sampling a subject's biometric and storing it as a **reference profile** (reference template). Traits that change (voice, facial hair, signature) need re-enrollment.

Throughput rate

The time the system needs to scan and process an enrolled subject.

Acceptance

How users view privacy, and physical or psychological discomfort.

TOKENS · SOMETHING YOU HAVE

Tokens and modern second factors

Token: a device/application the subject carries that stores or generates a credential.

Tokens are commonly combined with another factor (MFA).

Token type	How it works
Static	Holds a fixed value; mostly used for identification Smart card, swipe card
Synchronous dynamic	New password at fixed time intervals; clocks synced Authenticator app
Asynchronous dynamic	New password on an event, e.g. a button press
Challenge-response	System sends a challenge; token computes the response

SINGLE SIGN-ON

Single sign-on (SSO)

SSO: a subject is authenticated **once**, then accesses resource after resource without further authentication prompts.

Advantages

- Supports stronger passwords: users memorize only one
- Easier administration: the account is defined in fewer places

Main disadvantage

- If the SSO account is compromised, every connected service the user can reach may be exposed
- So SSO must be protected with MFA

Organizations agree to trust one **identity provider**, so a user logs in once and is accepted by services in other organizations.

PART B

Access control techniques

Once the subject is authenticated, what is it authorized to do? DAC · MAC · lattice · RBAC · rule-based · ABAC

ACCESS CONTROL TECHNIQUES

Discretionary vs nondiscretionary

Authorization happens only after authentication. Systems provide it through **access controls**, which manage the type and extent of access subjects have to objects.

Discretionary

The **owner** of an object decides who may access it, based on the subject's identity.

Model:

DAC

Nondiscretionary

A **central set of rules** governs the whole environment.

Access is not based on owner discretion or on user identity.

Models:

MAC, lattice-based, RBAC, TBAC, rule-based, ABAC

DISCRETIONARY ACCESS CONTROL

DAC: the owner decides

DAC: the owner or creator of an object controls and defines which subjects may access it. Access is based on the **identity** of the subject.

How it is implemented

With **access control lists (ACLs)** on each object, listing the access granted or denied to users and groups.

Strength

Flexible and simple for users.

Weakness

Users can grant access by mistake;

malware runs with the user's rights.

Example: a user creates a spreadsheet, becomes its owner, and changes its permissions to let a colleague read it.

ACCESS CONTROL MATRIX

ACL vs capability table

Subject	File1	File2	Printer
Ahmed	Read, write	Read	Print
Omar	Read	None	Print
Sara	None	Read, write	None

ACL = one column

Bound to an **object**. "Who can use File1?" Ahmed: read, write; Omar: read.

Capability table = one row

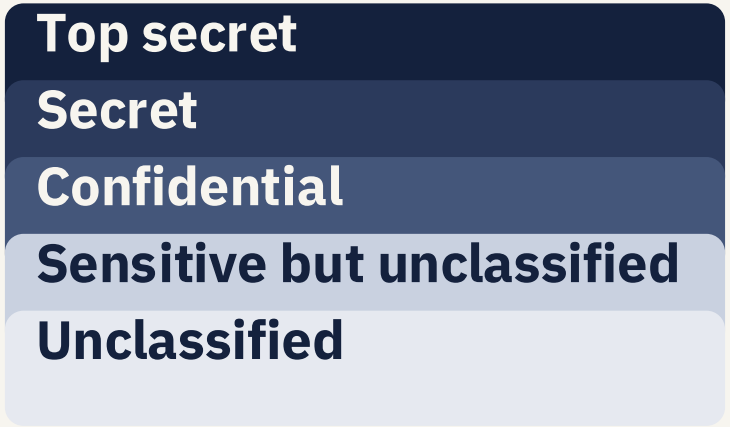
Bound to a **subject**. "What can Ahmed use?" File1, File2, Printer.

MANDATORY ACCESS CONTROL

MAC: labels and clearances decide

- Relies on **classification labels**. Subjects are labeled by **clearance**; objects by **classification**.
- **Clearance** = the security level assigned to a user.
- **Classification** = the security level assigned to data.
- Subjects access objects at the same or a lower level, and only if their work requires it (need to know)
- Each label is a **security domain**: a protected area of information governed by the same security rules.
- Centrally enforced, so owners cannot override it; less flexible and scalable than DAC.

Military labels, highest first



Commercial: confidential, proprietary, private, sensitive, public

MANDATORY ACCESS CONTROL

MAC: labels and clearances decide

These three describe how security levels or security areas can be organized inside a Mandatory Access Control system.

Hierarchical: The security levels are arranged in order from low to high.
Example: Public < Sensitive < Confidential < Secret < Top Secret
A user with a higher clearance may be allowed to access lower levels, depending on the policy.
Hierarchical = ordered security levels.

Compartmentalized: The system is divided into separate security compartments or areas.
Example: Project A,B&C: A user may have clearance for Project A but not Project B, even if both projects have the same security level.
Compartmentalized = separate security areas; access requires permission for the specific compartment.

MANDATORY ACCESS CONTROL

MAC: labels and clearances decide

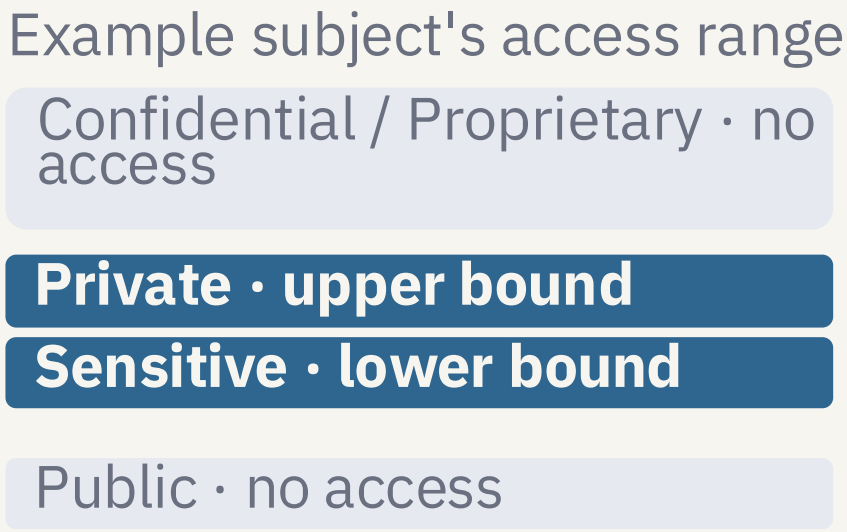
These three describe how security levels or security areas can be organized inside a Mandatory Access Control system.

Hybrid: This combines both approaches. You have security levels, and inside each level there are separate compartments.
Example:
Secret Project A & B
Top Secret Project A & C
A user may have (Clearance = Secret) and permission for (Project A)
So they can access Secret information in Project A, but not necessarily Secret information in Project B.
So:
Hybrid = hierarchical levels + separate compartments.

LATTICE-BASED ACCESS CONTROL

Lattice-based: upper and lower bounds

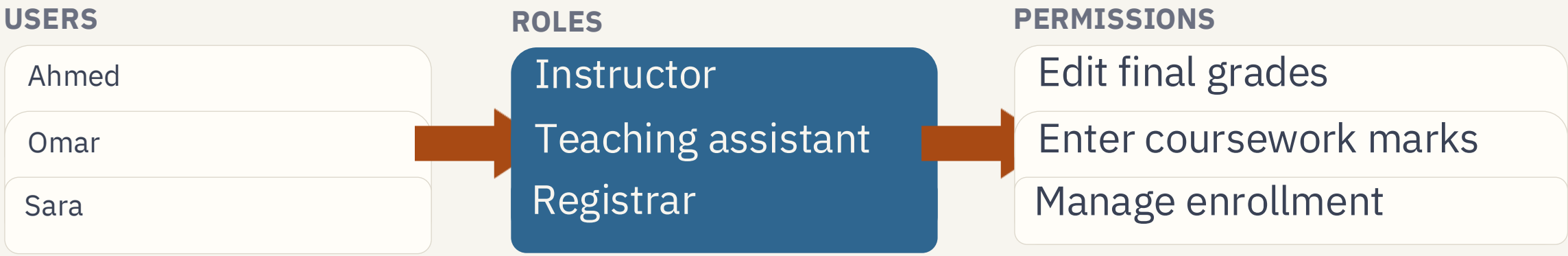
- **Lattice-based access control:** control organizes security labels in a structured order and controls which information a user can access based on the user's position in that structure
- **Lattice-based access control** defines an upper and a lower bound of access for every relationship between a subject and an object.
- Each subject gets a **least upper bound** and a **greatest lower bound** based on its position in the lattice.
- Developed to control information flow (confidentiality). MAC is a common example.



ROLE-BASED ACCESS CONTROL

RBAC: permissions follow the job

RBAC: access is defined by the subject's **role** (job description). **TBAC** (task-based) works the same way, using assigned **tasks** instead of a role.



Groups in DAC	Roles in RBAC (formal model)
Owner Decides	Job Role Decides Permissions
User can also get rights directly	Rights come only through roles
Access still based on owner discretion and identity	Access based on the job, not the person

RULE-BASED ACCESS CONTROL

Rule-based: the same rules for everyone

Rule-Based Access Control grants or denies access according to predefined system rules, such as time, network, device, or security conditions.

- **Global rules** imposed on all users, whoever they are.
- Classic example: a **firewall** rule set.
- Other examples: "no logins outside 08:00–18:00", "block USB storage on lab PCs".
- One rule change applies everywhere at once.
- **Rule-based = access is controlled by predefined rules.**

Firewall rules, checked in order

```
1 ALLOW tcp any → web:443
2 ALLOW tcp lab-net → server:22
3 DENY any any → any
```

Rule 3 is the implicit deny: anything not allowed is blocked.

Do not confuse

RBAC usually means **role**-based. → Role = your job;
Role-based = Who are you in the organization?

Rule = the same rule for everyone.
Rule-based = What rule applies to this request?

ATTRIBUTE-BASED ACCESS CONTROL

ABAC: decisions from attributes

ABAC: access is granted by evaluating rules over **attributes** of the subject, the object, the action and the environment.

```
ALLOW read patient_record
IF user.role = "doctor"
AND user.ward = record.ward
AND time is within user.shift
```

Subject	**Object**	**Action**	**Environment**
role, department	File type, sensitivity	read, edit, delete	time, location, device

ABAC makes access decisions using multiple attributes about the user, resource, action, and environment.

SUMMARY OF PART B

Access control models compared

Model	Who decides	Based on	Main strength / weakness	Typical use
DAC	Object owner	Subject identity; ACLs	Flexible / owners may over-share	Shared files
MAC	System policy	Labels and clearances	Central enforcement / inflexible	Military, government
Lattice	System policy	Upper and lower bounds	Controls information flow	MAC systems
RBAC	Administrator	Job role	Easy with staff turnover	Most organizations
Rule-based	Administrator	Global rules for all	One change applies to all	Firewalls
ABAC	Policy engine	Many attributes	Most flexible / complex	Cloud, large enterprises

APPLY IT

Which model fits each scenario?

Scenario	Model	Why
Students share project files on Google Drive	DAC	Owner chooses
Intelligence agency documents	MAC	Labels must be enforced
Hospital with 2,000 staff in 40 job types	RBAC	Access follows the job
University network firewall	Rule-based	Same rules for all traffic
Doctor sees records only for their ward, during their shift	ABAC	Depends on many attributes